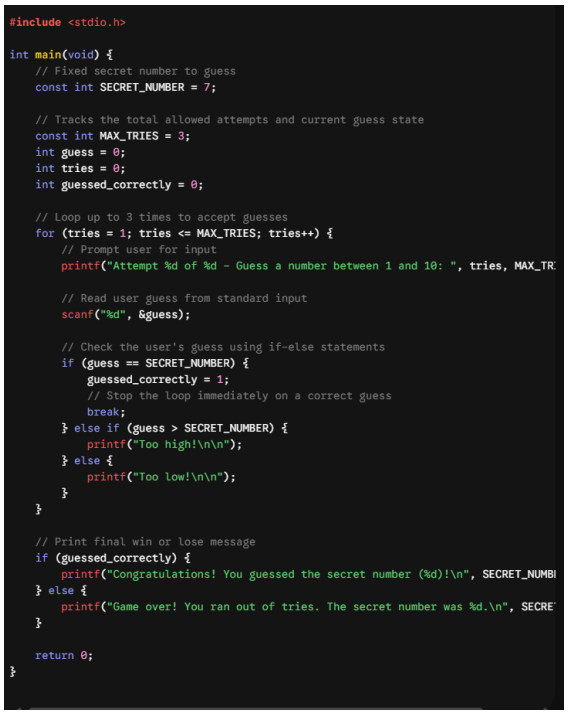
Gemini:



```c
#include <stdio.h>

int main(void) {
    // Fixed secret number to guess
    const int SECRET_NUMBER = 7;

    // Tracks the total allowed attempts and current guess state
    const int MAX_TRIES = 3;
    int guess = 0;
    int tries = 0;
    int guessed_correctly = 0;

    // Loop up to 3 times to accept guesses
    for (tries = 1; tries <= MAX_TRIES; tries++) {
        // Prompt user for input
        printf("Attempt %d of %d - Guess a number between 1 and 10: ", tries, MAX_TR

        // Read user guess from standard input
        scanf("%d", &guess);

        // Check the user's guess using if-else statements
        if (guess == SECRET_NUMBER) {
            guessed_correctly = 1;
            // Stop the loop immediately on a correct guess
            break;
        } else if (guess > SECRET_NUMBER) {
            printf("Too high!\n\n");
        } else {
            printf("Too low!\n\n");
        }
    }

    // Print final win or lose message
    if (guessed_correctly) {
        printf("Congratulations! You guessed the secret number (%d)!\n", SECRET_NUMBI
    } else {
        printf("Game over! You ran out of tries. The secret number was %d.\n", SECRE
    }

    return 0;
}
```

Prompt: Write a C program for EECS 348 Assignment 1 that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check if the guess is too high, too low, or correct. Stop the loop immediately if the user guesses the correct number. Print a final win or lose message. Include comments explaining the code.

Access: Via web on my personal laptop

Claude:

```c
#include <stdio.h>

int main(void) {
    const int secretNumber = 7;   // Fixed secret number between 1 and 10
    const int maxTries = 3;       // Maximum number of guesses allowed
    int guess;                    // Stores the user's guess
    int won = 0;                  // Flag to track whether the user won

    printf("Welcome to the Number Guessing Game!\n");
    printf("Guess a number between 1 and 10. You have %d tries.\n", maxTries);

    // Loop allows up to maxTries attempts
    for (int attempt = 1; attempt <= maxTries; attempt++) {
        printf("\nAttempt %d of %d - Enter your guess: ", attempt, maxTries);
        scanf("%d", &guess);

        // Check the guess against the secret number
        if (guess == secretNumber) {
            printf("Congratulations! You guessed the correct number: %d\n", secretNumb
            won = 1;   // Mark that the user won
            break;     // Stop the loop immediately since the guess is correct
        } else if (guess > secretNumber) {
            printf("Too high! Try again.\n");
        } else {
            printf("Too low! Try again.\n");
        }
    }

    // Final message depending on whether the user won or lost
    if (!won) {
        printf("\nSorry, you didn't guess the number. The correct number was %d.\n", s
    } else {
        printf("You win!\n");
    }

    return 0;
}
```

Prompt: Write a C program for EECS 348 Assignment 1 that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number. Use printf to prompt the user and scanf to read their guess. Use a while or for loop to let the user guess up to 3 times. Use if-else statements to check if the guess is too high, too low, or correct. Stop the loop immediately if the user guesses the correct number. Print a final win or lose message. Include comments explaining the code.

Access: Claude via web - Personal laptop

Comparison & Analysis

Correctness: Both programs followed the exact prompt i gave them, they both picked a fixed number(secret #) that had to be 1-10. Allowed the users up to 3 guesses, both used printf to prompt and scanf to get the input. Both Gen AI also used an if-else statement to decide if the input was too low/high/correct. Once the user would guess the right number or would run out of attempts it would stop the loop and use one of the print functions.

The Claude program was chosen because its structure was a bit more clear than gemini's in terms of readability and the provided comments.

Execution Time: Claude and Gemini were both in O(1) time complexity, so regardless of input the program was set to run x certain amount of times, in this case 3 times unless guessed before. Comparing the two it seems no program had an upper leg on the other.

Space Complexity: Again, both Gen AI were in O(1), so they both had a fixed number of variables like the user's guess, # of attempts, and a var to track if the user had won or not. The amount of memory used here doesn't affect the amount of attempts you get to guess the secret number, the program is limited to 3 attempts. Neither program has an advantage over the other here.

Maintainability: Gemini had decent maintainability, very similar to claude, but claude had made itself standout in terms of comments, which we learned a week ago in lecture that people underestimate good comments, if this was a program for work, it may not get touched again for a couple years and its important the next programmer knows what is going on, it's not always going to be your code you look at.

Program Selection and Improvements

Program Selected: Claude was selected over Gemini for the main purpose of maintainability, the comments made it so much easier to read and understand what each line was doing, because we used the same exact prompt for both Gen AI's, we got very similar programs from both. But what made Claude stand out even a bit more than just the comments was the variable names, and how easy it is to tell which each variable is pointing to.

Improvements made: Once selecting the Claude program, we needed to look over all the code to see if there is anything at all that could be improved. Just because something is AI doesn't make it perfect and this is where we can use our human input to give it a bit more detail. The main change made was adding "input validation" at scanf so the program knows if you've entered something that is not a whole number. If an invalid input was entered now like ("abc"), it will tell you that there was a bad input entered, and to type in a number instead. As well as not using an attempt when you do this. Some comments were modified or added as well just to make it more readable for future reference.

Effects of improvement: The input validation improved the reliability of the program, the most crucial part of the program is that you get 3 guesses, so adding this makes sure that your invalid input is corrected (by letting you know to use a number) and can now process your guess. So even with invalid inputs, the game will continue until you correct it.

Nothing improvement wise changed the execution time or space complexity, as it is still O(1).

The comments did improve maintainability and will help any who reads it understand exactly what is happening at each line.